Generative and Agentic AI

Fine-Tuning Assignment

Multi-Label Emotion Classification

Task Description

In this assignment, you are required to fine-tune a pretrained **encoder-only Transformer model** to identify the emotions expressed in an English comment. The objective is to predict all applicable emotion labels for each input while satisfying the resource and training constraints specified below. The choice of pretrained model, classification head, fine-tuning method, and training configuration is yours.

The dataset for this task is **GoEmotions**, using the `simplified` configuration. It contains English Reddit comments with one or more labels drawn from **27 emotions and neutral**. Use the official splits without redistributing examples between them:

Split	Examples	Purpose
Training	43,410	Training model parameters
Validation	5,426	Model selection and decision-rule tuning
Test	5,427	Final evaluation

Label vocabulary: admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, optimism, pride, realization, relief, remorse, sadness, surprise, and neutral. Preserve the dataset's original label-ID mapping and annotations, including records in which neutral co-occurs with another label.

Expected predictions: This is a **multi-label** task. Each input must receive one or more unique labels, and different inputs may require different numbers of labels. The correct number of labels is not supplied during inference. Your model must determine both which labels apply and how many to return. Label order does not matter.

You are required to:

- Implement the complete preprocessing, fine-tuning, validation, checkpoint-saving, and inference pipeline in a notebook.
- Select the final checkpoint and all prediction rules using only the training and validation splits. Handle inputs for which the initial decision rule selects no labels.
- Save a complete inference checkpoint and demonstrate that it can be reloaded in a fresh session to predict labels without retraining.

Constraints and Submission

Major constraints:

- **Model family:** Use a pretrained encoder-only Transformer. Decoder-only and encoder–decoder models are not permitted. The final solution must adapt the encoder; training only a head on a completely frozen encoder is insufficient.
- **Compute:** Training and inference must fit on a single GPU with at most **16 GB VRAM**. Kaggle, Google Colab, or a local environment may be used. Multi-GPU execution is not permitted.
- **Data and starting weights:** Train only on the official training split. External training examples, synthetic or pseudo-labelled training data, and checkpoints already fine-tuned for emotion classification are not permitted.
- **Final inference:** Submit one final model. Ensembles, external prediction APIs, and stored text-to-label lookup tables are not permitted. Do not use test labels for any training, model selection, decision-rule tuning, or post-processing.

Deliverables: Submit the following two items:

- **One notebook (.ipynb):** Include dependency versions, data preparation, model construction, training, validation results, checkpoint saving, and a separate inference section. Record the base checkpoint identifier and revision, random seed, final configuration, and parameter counts. The inference section must load the submitted checkpoint in a fresh session without running training cells. Expose a `predict(texts)` function that accepts a list of strings and returns one list of unique label names per input, in the same input order.
- **One checkpoint archive (.zip):** Include the selected model weights, tokenizer files, model configuration, label mapping, and any saved prediction-rule settings. If using adapters, include the trained adapters and classification head, together with the exact base-model identifier and revision needed to reconstruct the model. Clearly identify the final checkpoint to be evaluated.

Make sure the notebook reloads the submitted checkpoint correctly. The evaluator must be able to obtain predictions without retraining.

How we will test your model: We will load the submitted checkpoint using the notebook's inference section and evaluate predictions on the **complete official test split**. All 28 labels, including neutral, will be evaluated. The official test split is publicly available; it must nevertheless remain unused for development. The checkpoint and prediction procedure must be fixed before final evaluation.

The **Macro-F1 score** will determine marks. For each label j, let TP_j, FP_j, and FN_j denote the true-positive, false-positive, and false-negative counts across the full test split. We compute

$$F_{1,j} = \frac{2TP_j}{2TP_j + FP_j + FN_j}, \qquad F_{1,\text{macro}} = \frac{1}{28}\sum_{j=1}^{28} F_{1,j}.$$

If a label's denominator is zero, its F1 is set to zero. All 28 labels remain in the average. Macro-F1 gives each label equal weight. Micro-F1 and per-label F1 may be reported for analysis but do not contribute to marks.

Grading: Marks will be awarded primarily based on the Macro-F1 score obtained on the evaluator's test dataset, with higher scores receiving higher marks. No separate marks will be awarded for written justifications or architectural choices. However, inconsistencies in architectural design, incorrect model implementations, or architectures fundamentally unsuitable for the multi-label classification task will attract penalties, irrespective of the achieved Macro-F1 score.